HTML SVG

A Practical, From-Zero Learning Guide

Part 9 — SVG Animation & Interaction

SVG becomes especially powerful when it responds to time, hover, clicks and user input. This part covers CSS animation, transitions, SVG/SMIL animation, JavaScript animation, path drawing and practical interactive graphics.

> **Goal:** Learn several animation techniques and understand when to use each one.

1. The Three Main Ways to Animate SVG

• CSS transitions and animations — simple, declarative and excellent for UI effects.

• SMIL SVG animation — SVG's built-in animation system.

• JavaScript — maximum control for interactive and dynamic graphics.

You can also combine these approaches.

2. SVG and CSS

```
<svg class="logo" viewBox="0 0 200 100">
  <circle cx="100" cy="50" r="30"/>
</svg>

<style>
.logo circle {
  transition: transform 0.5s;
  transform-origin: center;
}

.logo:hover circle {
  transform: scale(1.4);
}
</style>
```

CSS can target SVG elements just like HTML elements.

3. CSS transition

```
.circle {
  fill: royalblue;
  transition:
    fill 0.3s ease,
    transform 0.4s ease;
}

.circle:hover {
  fill: tomato;
  transform: scale(1.2);
}
```

A transition smoothly interpolates a change between states.

4. Common Timing Functions

• linear — constant speed.

• ease — starts and ends more gently.

• ease-in — slower at the beginning.

• ease-out — slower at the end.

• ease-in-out — slower at both ends.

• cubic-bezier(...) — custom timing curve.

5. CSS @keyframes

```
@keyframes pulse {
  0% {
    transform: scale(1);
  }

  50% {
```

```
    transform: scale(1.25);
  }

  100% {
    transform: scale(1);
  }
}

.pulse {
  animation:
    pulse 1.2s ease-in-out infinite;
}
```

Keyframes define multiple stages of an animation.

6. SVG CSS Animation Example

```
<svg viewBox="0 0 200 100">
  <circle class="ball"
          cx="100" cy="50" r="20"/>
</svg>

<style>
.ball {
  animation: bounce 1s ease-in-out infinite;
}

@keyframes bounce {
  0%, 100% {
    transform: translateY(0);
  }

  50% {
    transform: translateY(-25px);
  }
}
</style>
```

7. animation-duration and iteration

```
.shape {
  animation-name: spin;
  animation-duration: 2s;
  animation-iteration-count: infinite;
  animation-timing-function: linear;
}
```

Useful properties include animation-name, duration, timing-function, delay, iteration-count, direction, fill-mode and play-state.

8. animation-direction

```
.shape {
  animation:
    move 2s ease-in-out infinite alternate;
}
```

alternate makes consecutive iterations run forward and backward.

9. animation-fill-mode

```
.shape {
  animation:
    move 2s ease forwards;
}
```

forwards keeps the element at its final keyframe after the animation finishes.

10. Pausing an Animation

```
.paused {
  animation-play-state: paused;
}
```

JavaScript or another class can add/remove this state.

11. Transforming SVG Elements

```
@keyframes spin {
  from {
    transform: rotate(0deg);
  }

  to {
    transform: rotate(360deg);
  }
}
```

Transforms such as translate, rotate, scale and skew are commonly animated.

12. transform-origin

```
.gear {
  transform-origin: 50% 50%;
  animation: spin 2s linear infinite;
}
```

The origin determines the point around which a transform rotates or scales.

13. Stroke Drawing Animation

```
<path class="draw"
      d="M20 100 C80 20 180 20 280 100"
      fill="none"
      stroke="black"
      stroke-width="5"/>

<style>
.draw {
  stroke-dasharray: 1000;
  stroke-dashoffset: 1000;
  animation: draw 2s ease forwards;
}

@keyframes draw {
  to {
    stroke-dashoffset: 0;
  }
}
</style>
```

This creates the classic 'line being drawn' effect.

14. Measuring the Correct Dash Length

```
const path = document.querySelector(".draw");
const length = path.getTotalLength();

path.style.strokeDasharray = length;
path.style.strokeDashoffset = length;
```

Using the actual path length is more reliable than guessing a large number.

15. Drawing Animation with CSS Variables

```
.draw {
```

```
  --length: 500;
  stroke-dasharray: var(--length);
  stroke-dashoffset: var(--length);
  animation: draw 2s forwards;
}

@keyframes draw {
  to {
    stroke-dashoffset: 0;
  }
}
```

16. SVG SMIL

SVG also provides animation elements such as **<animate>**, **<animateTransform>** and **<animateMotion>**.

```
<circle cx="50" cy="50" r="20">
  <animate
    attributeName="cx"
    from="50"
    to="250"
    dur="2s"
    repeatCount="indefinite"/>
</circle>
```

SMIL is declared directly inside SVG.

17. animate attributeName

```
<circle cx="50" cy="50" r="20">
  <animate
    attributeName="r"
    from="20"
    to="40"
    dur="1s"
    repeatCount="indefinite"/>
</circle>
```

attributeName tells SVG which attribute is being animated.

18. values and keyTimes

```
<circle cx="100" cy="50" r="20">
  <animate
    attributeName="cx"
    values="50;150;250"
    keyTimes="0;0.5;1"
    dur="2s"
    repeatCount="indefinite"/>
</circle>
```

This lets you define multiple values throughout an animation.

19. begin and end

```
<circle cx="50" cy="50" r="20">
  <animate
    attributeName="cx"
    from="50"
    to="250"
    dur="2s"
    begin="click"/>
</circle>
```

SMIL supports timing triggers such as click and delayed start values.

20. animateTransform

```
<rect x="80" y="30"
```

```
      width="40"
      height="40">
  <animateTransform
    attributeName="transform"
    type="rotate"
    from="0 100 50"
    to="360 100 50"
    dur="2s"
    repeatCount="indefinite"/>
</rect>
```

animateTransform is useful for rotations, translations, scaling and other transforms.

21. animateMotion

```
<circle r="10">
  <animateMotion
    path="M20 100 C100 20 200 20 280 100"
    dur="3s"
    repeatCount="indefinite"/>
</circle>
```

animateMotion moves an element along a path.

22. Motion Along a Path

```
<path id="route"
      d="M30 150
         C100 20 200 20 270 150"
      fill="none"
      stroke="lightgray"/>

<circle r="8">
  <animateMotion
    dur="3s"
    repeatCount="indefinite">
    <mpath href="#route"/>
  </animateMotion>
</circle>
```

Using an existing path with mpath is useful when the same route is visible and animated.

23. JavaScript Animation

```
const circle = document.querySelector("#circle");

let x = 20;

function animate() {
  x += 1;
  circle.setAttribute("cx", x);

  if (x < 280) {
    requestAnimationFrame(animate);
  }
}

requestAnimationFrame(animate);
```

requestAnimationFrame() asks the browser to run the next animation step at an appropriate time for rendering.

24. requestAnimationFrame

For interactive browser animations, requestAnimationFrame is generally preferable to a tight setInterval loop because it integrates with the browser's rendering cycle.

25. Time-Based Animation

```
let start;

function animate(timestamp) {
  if (!start) start = timestamp;

  const elapsed = timestamp - start;
  const progress = Math.min(elapsed / 2000, 1);

  const x = 20 + 260 * progress;

  circle.setAttribute("cx", x);

  if (progress < 1) {
    requestAnimationFrame(animate);
  }
}

requestAnimationFrame(animate);
```

Time-based animation is better than simply adding a fixed amount every frame because frame rates can vary.

26. Easing in JavaScript

```
function easeOut(t) {
  return 1 - Math.pow(1 - t, 3);
}

const eased = easeOut(progress);
```

Easing changes how progress is distributed over time.

27. Hover Interaction

```
<svg class="card" viewBox="0 0 200 120">
  <rect x="20" y="20"
        width="160" height="80"
        rx="15"/>
</svg>

<style>
.card rect {
  transition:
    transform 0.3s,
    fill 0.3s;
}

.card:hover rect {
  transform: translateY(-8px);
  fill: tomato;
}
</style>
```

28. Click Interaction

```
<svg id="toggle" viewBox="0 0 200 100">
  <circle id="button"
          cx="50" cy="50" r="30"/>
</svg>

<script>
const button = document.querySelector("#button");

button.addEventListener("click", () => {
  button.classList.toggle("active");
});
</script>

<style>
#button {
  fill: gray;
}
```

```
#button.active {
  fill: limegreen;
}
</style>
```

29. Interactive Progress Ring

```
<circle
  id="ring"
  cx="60" cy="60" r="45"
  fill="none"
  stroke="lightgray"
  stroke-width="10"/>

<circle
  id="progress"
  cx="60" cy="60" r="45"
  fill="none"
  stroke="royalblue"
  stroke-width="10"
  transform="rotate(-90 60 60)"
  stroke-linecap="round"/>
```

JavaScript can calculate the circumference and update stroke-dashoffset to represent progress.

30. Progress Ring Formula

```
const radius = 45;
const circumference = 2 * Math.PI * radius;

progress.style.strokeDasharray = circumference;

function setProgress(percent) {
  const offset =
    circumference * (1 - percent / 100);

  progress.style.strokeDashoffset = offset;
}
```

31. Interactive Tooltip

SVG elements can respond to pointer events. A simple tooltip can read data attributes from the target and position an HTML element near the SVG.

```
<circle
  class="point"
  cx="100"
  cy="80"
  r="8"
  data-value="42"/>

<script>
document.querySelectorAll(".point")
  .forEach(point => {
    point.addEventListener("mouseenter", () => {
      console.log(point.dataset.value);
    });
  });
</script>
```

32. pointer-events

```
.overlay {
  pointer-events: none;
}
```

The pointer-events property controls whether SVG elements participate in pointer hit testing. This is useful when an invisible layer should not block interaction.

33. Clickable Paths

```
<path
  d="M20 20 H180 V100 H20 Z"
  fill="lightblue"
  tabindex="0"
  role="button"/>
```

For keyboard-accessible interactive SVG, consider focusability, keyboard interaction and appropriate semantics instead of relying only on mouse events.

34. prefers-reduced-motion

```
@media (prefers-reduced-motion: reduce) {
  .animated {
    animation: none;
    transition: none;
  }
}
```

Respecting reduced-motion preferences is important for accessible motion.

35. CSS vs SMIL vs JavaScript

Technique	Best for	Strength
CSS	Hover, UI states, looping effects	Simple and maintainable
SMIL	SVG-native animation	Direct SVG timing and motion
JavaScript	Interactive/dynamic graphics	Maximum control

36. Animating fill and stroke

```
.icon {
  fill: transparent;
  stroke: black;
  transition:
    fill 0.3s,
    stroke 0.3s;
}

.icon:hover {
  fill: gold;
  stroke: orange;
}
```

37. Animated Loading Spinner

```
<svg class="spinner"
     viewBox="0 0 100 100">
  <circle
    cx="50" cy="50" r="35"
    fill="none"
    stroke="currentColor"
    stroke-width="8"
    stroke-linecap="round"
    stroke-dasharray="80 140"/>
</svg>

<style>
.spinner {
  animation: spin 1s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
```

```
  }
}
</style>
```

38. Animated Hamburger Icon

```
<svg class="menu"
     viewBox="0 0 100 80">
  <path class="top" d="M10 15 H90"/>
  <path class="middle" d="M10 40 H90"/>
  <path class="bottom" d="M10 65 H90"/>
</svg>

<style>
.menu path {
  stroke: black;
  stroke-width: 8;
  stroke-linecap: round;
  transition: transform .3s, opacity .3s;
}

.menu.open .top {
  transform: translateY(25px) rotate(45deg);
}

.menu.open .middle {
  opacity: 0;
}

.menu.open .bottom {
  transform: translateY(-25px) rotate(-45deg);
}
</style>
```

39. Animated Logo Idea

• Draw the logo as paths.

• Use stroke-drawing to reveal it.

• Add a small scale or opacity effect.

• Use currentColor so it can follow the page theme.

• Keep the animation short enough that it does not delay the interface.

40. Animation Performance

• Prefer transform and opacity for many UI animations.

• Avoid animating huge numbers of complex paths simultaneously.

• Keep filters and expensive effects under control.

• Use requestAnimationFrame for JavaScript-driven visual updates.

• Do not update the DOM unnecessarily on every frame.

• Test animations on slower devices.

41. Common Animation Mistakes

• Using setInterval for frame-by-frame rendering.

• Animating too many complex elements.

• Leaving infinite animations running when they are not needed.

• Making animations too long.

• Ignoring reduced-motion preferences.

• Using transform origins incorrectly.

• Guessing path lengths instead of measuring them.

• Using animation for decoration that distracts from important content.

42. Mini Project — Drawing Logo

```
<svg viewBox="0 0 300 150">
  <path class="logo-line"
        d="M30 110
           C80 20 170 20 270 110"
        fill="none"
        stroke="currentColor"
        stroke-width="7"
        stroke-linecap="round"/>
</svg>

<style>
.logo-line {
  stroke-dasharray: 500;
  stroke-dashoffset: 500;
  animation: reveal 2s ease forwards;
}

@keyframes reveal {
  to {
    stroke-dashoffset: 0;
  }
}
</style>
```

43. Mini Project — Bouncing Ball

```
<svg viewBox="0 0 300 180">
  <circle class="ball"
          cx="150" cy="50" r="18"/>
</svg>

<style>
.ball {
  animation: bounce 1s ease-in-out infinite;
}

@keyframes bounce {
  0%, 100% {
    transform: translateY(90px);
  }

  50% {
    transform: translateY(0);
  }
}
</style>
```

44. Mini Project — Path Motion

```
<svg viewBox="0 0 400 200">
  <path id="track"
        d="M30 150
           C100 20 300 20 370 150"
        fill="none"
        stroke="lightgray"/>

  <circle r="10">
    <animateMotion
      dur="3s"
```

```
      repeatCount="indefinite">
      <mpath href="#track"/>
    </animateMotion>
  </circle>
</svg>
```

45. Mini Project — Progress Indicator

```
<svg viewBox="0 0 120 120">
  <circle cx="60" cy="60" r="45"
          fill="none"
          stroke="lightgray"
          stroke-width="10"/>

  <circle id="progress"
          cx="60" cy="60" r="45"
          fill="none"
          stroke="currentColor"
          stroke-width="10"
          stroke-linecap="round"
          transform="rotate(-90 60 60)"/>
</svg>

<script>
const p = document.querySelector("#progress");
const c = 2 * Math.PI * 45;

p.style.strokeDasharray = c;
p.style.strokeDashoffset = c;

setTimeout(() => {
  p.style.strokeDashoffset = c * 0.25;
}, 100);
</script>
```

46. Exercises

• Make a circle pulse continuously.

• Create a spinning gear.

• Animate a path so it appears to draw itself.

• Create a bouncing ball with CSS.

• Move a circle along a curved path.

• Build an animated loading spinner.

• Build an interactive progress ring.

• Create a hamburger icon that transforms into an X.

• Make an SVG card respond to hover.

• Create a clickable SVG button with an active state.

• Add reduced-motion support to your animations.

• Build an animated logo using only SVG paths and CSS.

47. Quick Reference

Tool	Use
transition	Smooth state changes
@keyframes	Multi-step CSS animation

animation-*	Control CSS animations
stroke-dasharray	Creates dash pattern
stroke-dashoffset	Offsets dash pattern; useful for drawing effects
requestAnimationFrame()	Browser-friendly JS animation loop
getTotalLength()	Measure SVG path
getPointAtLength()	Find a point on a path
<animate>	SMIL attribute animation
<animateTransform>	SMIL transform animation
<animateMotion>	SMIL path motion
<mpath>	References a path for motion
pointer-events	Controls pointer hit testing
prefers-reduced-motion	Respect motion preferences

48. What's Next?

Part 10 will cover **SVG with HTML, CSS, JavaScript and React**: embedding SVG, external files, inline SVG, CSS targeting, SVG sprites in real projects, React components, props, events, dynamic SVG, responsive SVG and production patterns.